JANGAM PALLAVI

23BTRCM024

JAIN(Deemed-to-be-University)

CLOUD TECHNOLOGY AND MOBILE APPLICATION

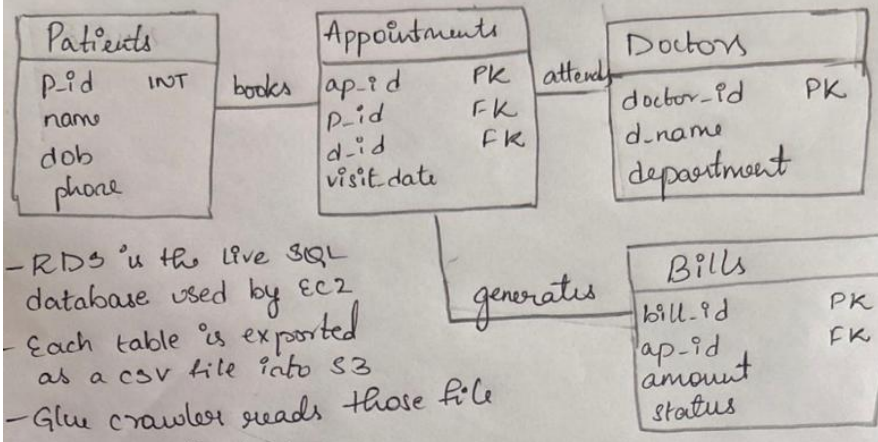
2023 - 2027

Date : 10/10/2026

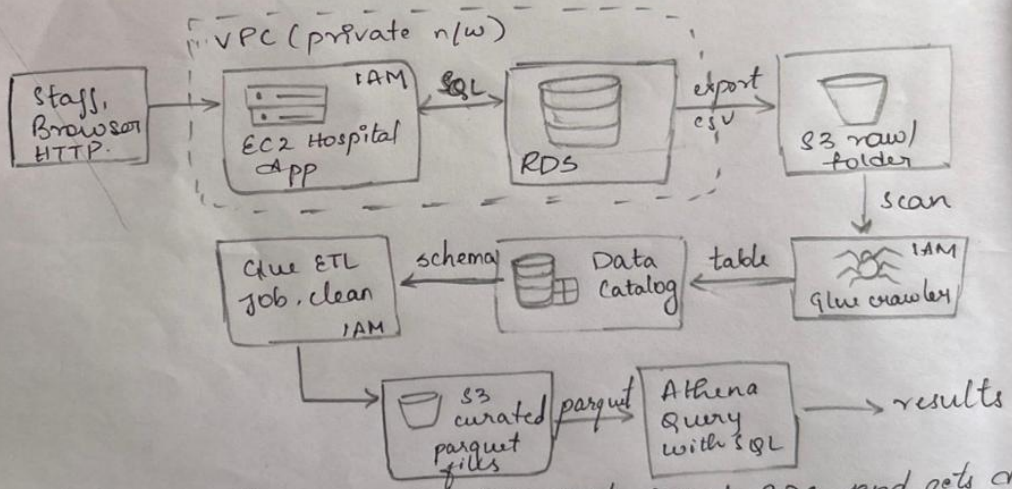
TITLE : AWS Cloud & Data Engineering

Hospital Management

Database:



- RDS is the live SQL database used by EC2
- Each table is exported as a CSV file into S3
- Glue crawler reads those file and create tables.



- Staff opens the app on EC2. EC2 sends SQL to RDS and gets data back.
- RDS exports its tables as CSV files into S3 raw.
- The Glue crawler scans S3 and writes the table details into the data catalog.
- The Glue ETL job reads the files, uses the schema from the Data Catalog, and writes clean parquet to S3 curated.
- Athena queries S3 curated, and show the results as dashboard.

Hospital Management System Using SQL

```sql
create database hospitaldb;
use hospitaldb;

-- PATIENTS TABLE --
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    patient_name VARCHAR(50),
    age INT,
    gender VARCHAR(10),
    city VARCHAR(50),
    admission_date DATE,
    doctor_id INT
);
```

```sql
INSERT INTO Patients VALUES
(1, 'Aarav', 25, 'Male', 'Bengaluru', '2026-01-10', 101),
(2, 'Priya', 32, 'Female', 'Hyderabad', '2026-01-15', 102),
(3, 'Rahul', 45, 'Male', 'Chennai', '2026-02-05', 103),
(4, 'Sneha', 28, 'Female', 'Mumbai', '2026-02-12', 101),
(5, 'Arjun', 60, 'Male', 'Pune', '2026-03-01', 104),
(6, 'Divya', 38, 'Female', 'Bengaluru', '2026-03-10', 102),
(7, 'Kiran', 52, 'Male', 'Mysuru', '2026-04-08', 105),
(8, 'Ananya', 19, 'Female', 'Chennai', '2026-04-20', 103),
(9, 'Vikram', 67, 'Male', 'Hyderabad', '2026-05-05', 104),
(10, 'Meera', 41, 'Female', 'Mumbai', '2026-05-18', 105);

select * from Patients;
```

Result Grid

patient_id	patient_name	age	gender	city	admission_date	doctor_id
1	Aarav	25	Male	Bengaluru	2026-01-10	101
2	Priya	32	Female	Hyderabad	2026-01-15	102
3	Rahul	45	Male	Chennai	2026-02-05	103
4	Sneha	28	Female	Mumbai	2026-02-12	101
5	Arjun	60	Male	Pune	2026-03-01	104
6	Divya	38	Female	Bengaluru	2026-03-10	102
7	Kiran	52	Male	Mysuru	2026-04-08	105
8	Ananya	19	Female	Chennai	2026-04-20	103
9	Vikram	67	Male	Hyderabad	2026-05-05	104
10	Meera	41	Female	Mumbai	2026-05-18	105
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Patients 5

```sql
-- DOCTOR TABLE --
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    doctor_name VARCHAR(50),
    specialization VARCHAR(50),
    experience INT,
    consultation_fee DECIMAL(10,2)
);

INSERT INTO Doctors VALUES
(101, 'Dr. Anil Kumar', 'Cardiology', 12, 800.00),
(102, 'Dr. Priya Sharma', 'Dermatology', 8, 600.00),
(103, 'Dr. Ravi Patel', 'Neurology', 15, 1200.00),
(104, 'Dr. Sneha Reddy', 'Orthopedics', 10, 900.00),
(105, 'Dr. Kiran Rao', 'Pediatrics', 7, 500.00),
(106, 'Dr. Meera Nair', 'Gynecology', 11, 1000.00),
(107, 'Dr. Arjun Singh', 'Cardiology', 6, 700.00),
(108, 'Dr. Divya Menon', 'Dermatology', 5, 450.00),
(109, 'Dr. Vikram Das', 'Neurology', 9, 950.00),
(110, 'Dr. Ananya Gupta', 'Orthopedics', 14, 1100.00);

Select * from Doctors;
```

```sql
Select * from Doctors;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

doctor_id	doctor_name	specialization	experience	consultation_fee
101	Dr. Anil Kumar	Cardiology	12	800.00
102	Dr. Priya Sharma	Dermatology	8	600.00
103	Dr. Ravi Patel	Neurology	15	1200.00
104	Dr. Sneha Reddy	Orthopedics	10	900.00
105	Dr. Kiran Rao	Pediatrics	7	500.00
106	Dr. Meera Nair	Gynecology	11	1000.00
107	Dr. Arjun Singh	Cardiology	6	700.00
108	Dr. Divya Menon	Dermatology	5	450.00
109	Dr. Vikram Das	Neurology	9	950.00
110	Dr. Ananya Gupta	Orthopedics	14	1100.00
NULL	NULL	NULL	NULL	NULL

Doctors 6

```sql
-- BILLING TABLE --
CREATE TABLE Billing (
    bill_id INT PRIMARY KEY,
    patient_id INT,
    bill_date DATE,
    amount DECIMAL(10,2),
    payment_status VARCHAR(20),
    payment_method VARCHAR(20),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

INSERT INTO Billing VALUES
(1, 1, '2026-01-12', 5000.00, 'Paid', 'UPI'),
(2, 2, '2026-01-18', 7500.00, 'Pending', 'Card'),
(3, 3, '2026-02-08', 12000.00, 'Paid', 'Cash'),
(4, 4, '2026-02-15', 6500.00, 'Pending', 'UPI'),
(5, 5, '2026-03-05', 15000.00, 'Paid', 'Insurance'),
(6, 6, '2026-03-12', 8200.00, 'Paid', 'Card'),
(7, 7, '2026-04-10', 9500.00, 'Pending', 'Insurance'),
(8, 8, '2026-04-22', 4000.00, 'Paid', 'Cash'),
(9, 9, '2026-05-08', 18000.00, 'Pending', 'Card'),
(10, 10, '2026-05-20', 6800.00, 'Paid', 'UPI');

Select * from Billing;
```

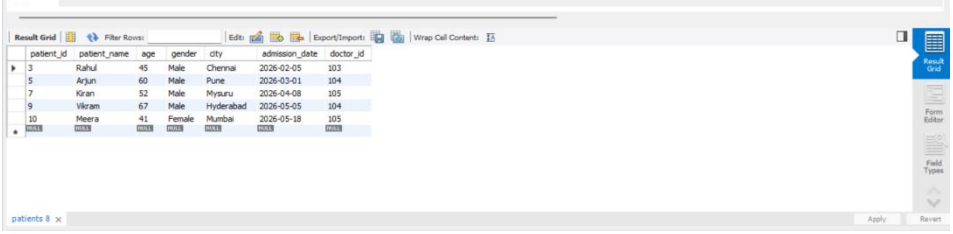
```sql
Select * from Billing;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

bill_id	patient_id	bill_date	amount	payment_status	payment_method
1	1	2026-01-12	5000.00	Paid	UPI
2	2	2026-01-18	7500.00	Pending	Card
3	3	2026-02-08	12000.00	Paid	Cash
4	4	2026-02-15	6500.00	Pending	UPI
5	5	2026-03-05	15000.00	Paid	Insurance
6	6	2026-03-12	8200.00	Paid	Card
7	7	2026-04-10	9500.00	Pending	Insurance
8	8	2026-04-22	4000.00	Paid	Cash
9	9	2026-05-08	18000.00	Pending	Card
10	10	2026-05-20	6800.00	Paid	UPI
NULL	NULL	NULL	NULL	NULL	NULL

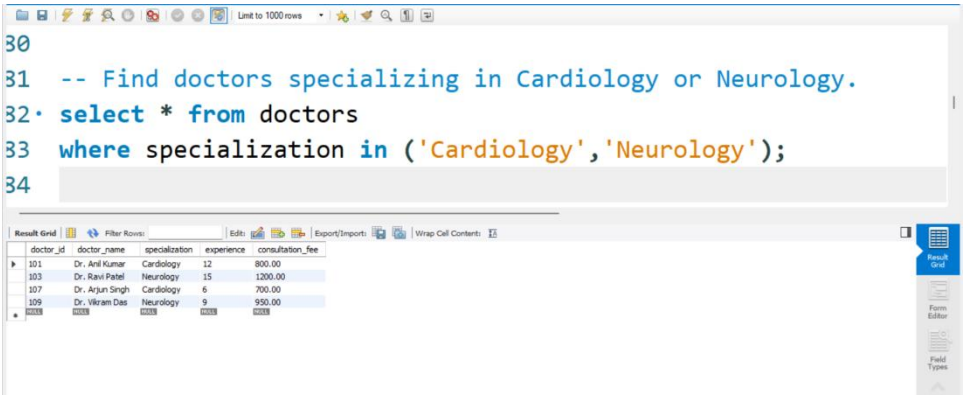
Billing 7

```sql
-- Display patient details for patients older than 40.--
select * from patients
where age > 40;
```



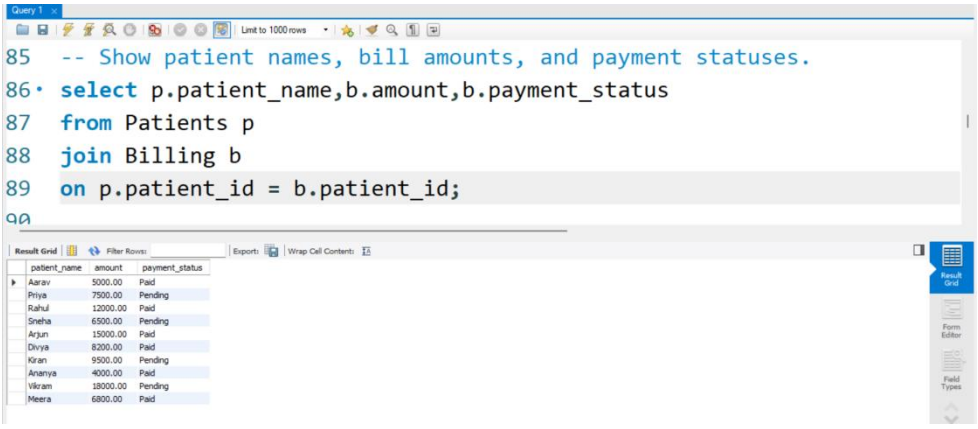
patient_id	patient_name	age	gender	city	admission_date	doctor_id
3	Rahul	45	Male	Chennai	2026-02-05	103
5	Arjun	60	Male	Pune	2026-03-01	104
7	Kiran	52	Male	Mysuru	2026-04-08	105
9	Vikram	67	Male	Hyderabad	2026-05-05	104
10	Meera	41	Female	Mumbai	2026-05-18	105

```sql
-- Find doctors specializing in Cardiology or Neurology.
select * from doctors
where specialization in ('Cardiology','Neurology');
```



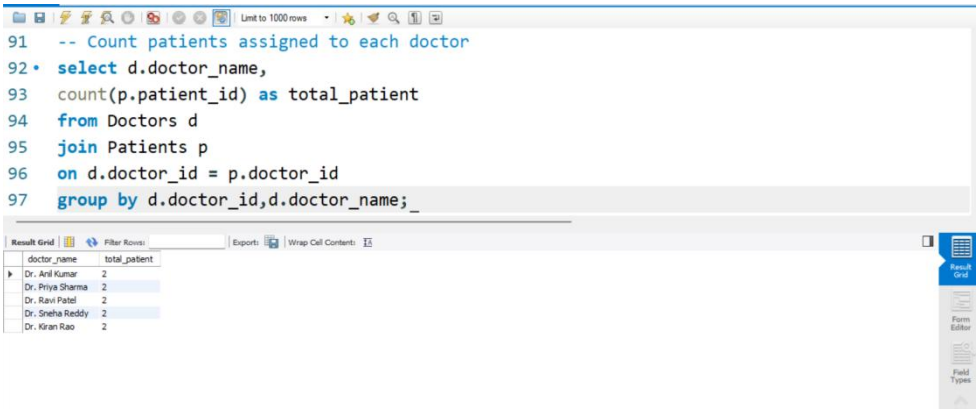
doctor_id	doctor_name	specialization	experience	consultation_fee
101	Dr. Anil Kumar	Cardiology	12	800.00
103	Dr. Ravi Patel	Neurology	15	1200.00
107	Dr. Arjun Singh	Cardiology	6	700.00
109	Dr. Vikram Das	Neurology	9	950.00

```sql
-- Show patient names, bill amounts, and payment statuses.
select p.patient_name,b.amount,b.payment_status
from Patients p
join Billing b
on p.patient_id = b.patient_id;
```



patient_name	amount	payment_status
Aarav	5000.00	Paid
Priya	7500.00	Pending
Rahul	12000.00	Paid
Sneha	6500.00	Pending
Arjun	15000.00	Paid
Divya	8200.00	Paid
Kiran	9500.00	Pending
Ananya	4000.00	Paid
Vikram	18000.00	Pending
Meera	6800.00	Paid

```sql
-- Count patients assigned to each doctor
select d.doctor_name,
count(p.patient_id) as total_patient
from Doctors d
join Patients p
on d.doctor_id = p.doctor_id
group by d.doctor_id,d.doctor_name;
```



doctor_name	total_patient
Dr. Anil Kumar	2
Dr. Priya Sharma	2
Dr. Ravi Patel	2
Dr. Sneha Reddy	2
Dr. Kiran Rao	2

```sql
-- Find doctors with more than one patient
SELECT d.doctor_name,COUNT(p.patient_id) AS total_patients
FROM Doctors d
JOIN Patients p
ON d.doctor_id = p.doctor_id
GROUP BY d.doctor_id, d.doctor_name
HAVING COUNT(p.patient_id) > 1;
```

doctor_name	total_patients
Dr. Anil Kumar	2
Dr. Priya Sharma	2
Dr. Ravi Patel	2
Dr. Sneha Reddy	2
Dr. Kiran Rao	2

```sql
-- Find bills above the average bill amount
SELECT p.patient_name, b.amount
FROM Patients p
JOIN Billing b
ON p.patient_id = b.patient_id
WHERE b.amount > (
SELECT AVG(amount)
    FROM Billing);
```

patient_name	amount
Rahul	12000.00
Arjun	15000.00
Kiran	9500.00
Vikram	18000.00

```sql
-- Calculate revenue by doctor
SELECT d.doctor_name, SUM(b.amount) AS total_revenue
FROM Doctors d
JOIN Patients p
ON d.doctor_id = p.doctor_id
JOIN Billing b
ON p.patient_id = b.patient_id
GROUP BY d.doctor_id, d.doctor_name
ORDER BY total_revenue DESC;
```

doctor_name	total_revenue
Dr. Sneha Reddy	33000.00
Dr. Kiran Rao	16300.00
Dr. Ravi Patel	16000.00
Dr. Priya Sharma	15700.00
Dr. Anil Kumar	11500.00

```sql
-- Find patients with high bills and pending payments
SELECT p.patient_name, p.city, b.amount, b.payment_status
FROM Patients p
JOIN Billing b
ON p.patient_id = b.patient_id
WHERE b.amount > 10000
AND b.payment_status = 'Pending';
```

patient_name	city	amount	payment_status
Vikram	Hyderabad	18000.00	Pending

```sql
-- Find patients whose bills are above ₹15,000 using a CTE
WITH HighBills AS (
    SELECT patient_id, amount
    FROM Billing
    WHERE amount > 15000
)
SELECT p.patient_name, hb.amount
FROM HighBills hb
JOIN Patients p
ON hb.patient_id = p.patient_id;
```

Result Grid

patient_name	amount
Vikram	18000.00

```sql
-- Classify hospital bills into categories based on their amount using a CASE statement.
SELECT bill_id, amount,
    CASE
        WHEN amount >= 20000 THEN 'High'
        WHEN amount >= 10000 THEN 'Medium'
        ELSE 'Low'
    END AS bill_category
FROM Billing;
```

Result Grid

bill_id	amount	bill_category
1	5000.00	Low
2	7500.00	Low
3	12000.00	Medium
4	6500.00	Low
5	15000.00	Medium
6	8200.00	Low
7	9500.00	Low
8	4000.00	Low
9	18000.00	Medium
10	6800.00	Low

```sql
-- Rank doctors by number of patients
WITH DoctorCounts AS (
    SELECT d.doctor_id, d.doctor_name, COUNT(p.patient_id) AS total_patients
    FROM Doctors d
    LEFT JOIN Patients p
        ON d.doctor_id = p.doctor_id
    GROUP BY d.doctor_id, d.doctor_name
)
SELECT doctor_name, total_patients,
RANK() OVER (
        ORDER BY total_patients DESC) AS doctor_rank
```

Result Grid

doctor_name	total_patients	doctor_rank
Dr. Anil Kumar	2	1
Dr. Priya Sharma	2	1
Dr. Ravi Patel	2	1
Dr. Sneha Reddy	2	1
Dr. Kiran Rao	2	1
Dr. Meera Nair	0	6
Dr. Arjun Singh	0	6
Dr. Divya Menon	0	6
Dr. Vikram Das	0	6
Dr. Ananya Gupta	0	6

```sql
-- Compare each bill with the previous bill
SELECT bill_id, bill_date, amount,
LAG(amount) OVER (
        ORDER BY bill_date, bill_id
    ) AS previous_bill,
    amount - LAG(amount) OVER (
        ORDER BY bill_date, bill_id
    ) AS bill_difference
FROM Billing;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_id	bill_date	amount	previous_bill	bill_difference
1	2026-01-12	5000.00	NULL	NULL
2	2026-01-18	7500.00	5000.00	2500.00
3	2026-02-08	12000.00	7500.00	4500.00
4	2026-02-15	6500.00	12000.00	-5500.00
5	2026-03-05	15000.00	6500.00	8500.00
6	2026-03-12	8200.00	15000.00	-6800.00
7	2026-04-10	9500.00	8200.00	1300.00
8	2026-04-22	4000.00	9500.00	-5500.00
9	2026-05-08	18000.00	4000.00	14000.00
10	2026-05-20	6800.00	18000.00	-11200.00

```sql
-- Find the next bill amount
SELECT bill_id,bill_date, amount,
LEAD(amount) OVER (
        ORDER BY bill_date, bill_id
    ) AS next_bill_amount
FROM Billing;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_id	bill_date	amount	next_bill_amount
1	2026-01-12	5000.00	7500.00
2	2026-01-18	7500.00	12000.00
3	2026-02-08	12000.00	6500.00
4	2026-02-15	6500.00	15000.00
5	2026-03-05	15000.00	8200.00
6	2026-03-12	8200.00	9500.00
7	2026-04-10	9500.00	4000.00
8	2026-04-22	4000.00	18000.00
9	2026-05-08	18000.00	6800.00
10	2026-05-20	6800.00	NULL

```sql
-- Display all doctors, including doctors who have no patients assigned to them.
SELECT d.doctor_name, d.specialization,p.patient_name
FROM Doctors d
LEFT JOIN Patients p
ON d.doctor_id = p.doctor_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

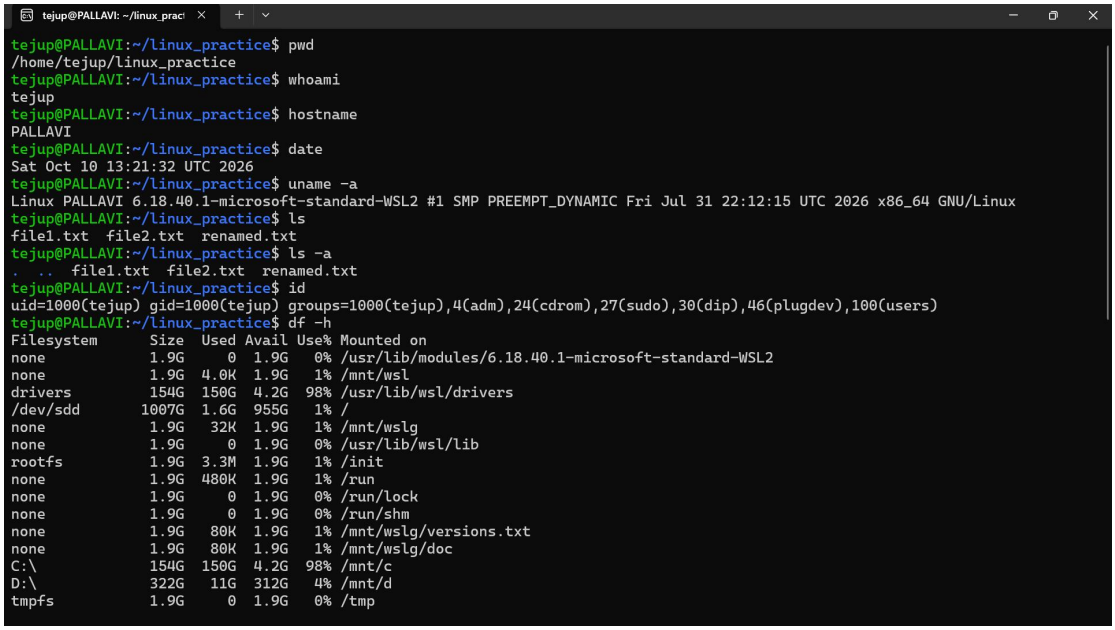
doctor_name	specialization	patient_name
Dr. Anil Kumar	Cardiology	Sneha
Dr. Anil Kumar	Cardiology	Aarav
Dr. Priya Sharma	Dermatology	Divya
Dr. Priya Sharma	Dermatology	Priya
Dr. Ravi Patel	Neurology	Ananya
Dr. Ravi Patel	Neurology	Rahul
Dr. Sneha Reddy	Orthopedics	Vikram
Dr. Sneha Reddy	Orthopedics	Arjun
Dr. Kiran Rao	Pediatrics	Meera
Dr. Kiran Rao	Pediatrics	Kiran
Dr. Meera Nair	Gynecology	NULL
Dr. Arjun Singh	Cardiology	NULL
Dr. Divya Menon	Dermatology	NULL
Dr. Vikram Das	Neurology	NULL
Dr. Ananya Gupta	Orthopedics	NULL

```sql
-- Count the number of patients assigned to each doctor.
SELECT d.doctor_name, COUNT(p.patient_id) AS total_patients
FROM Doctors d
LEFT JOIN Patients p
ON d.doctor_id = p.doctor_id
GROUP BY d.doctor_id, d.doctor_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

doctor_name	total_patients
Dr. Anil Kumar	2
Dr. Priya Sharma	2
Dr. Ravi Patel	2
Dr. Sneha Reddy	2
Dr. Kiran Rao	2
Dr. Meera Nair	0
Dr. Arjun Singh	0
Dr. Divya Menon	0
Dr. Vikram Das	0
Dr. Ananya Gupta	0

Linux Commands:



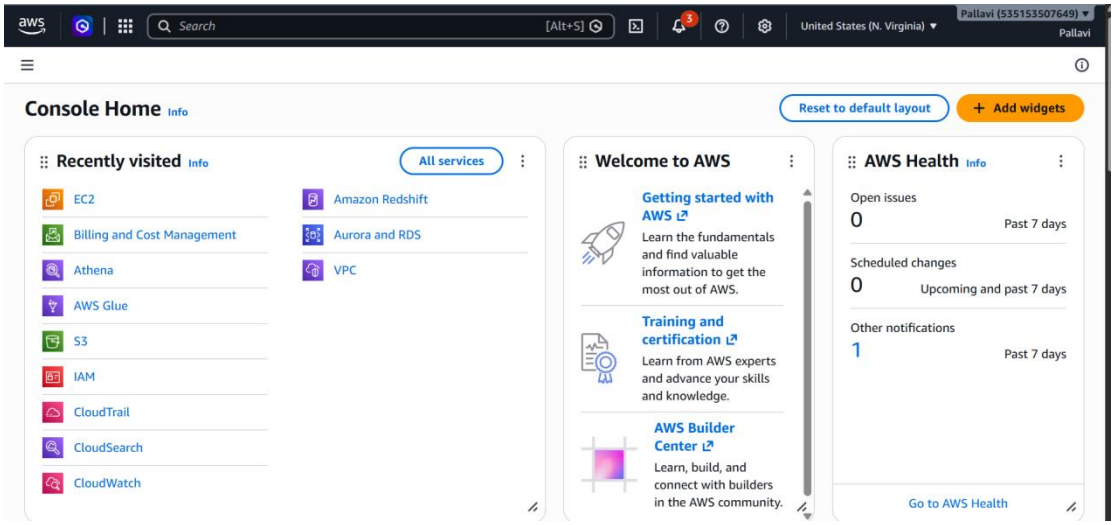
```
tejup@PALLAVI:~/linux_practice$ pwd
/home/tejup/linux_practice
tejup@PALLAVI:~/linux_practice$ whoami
tejup
tejup@PALLAVI:~/linux_practice$ hostname
PALLAVI
tejup@PALLAVI:~/linux_practice$ date
Sat Oct 10 13:21:32 UTC 2026
tejup@PALLAVI:~/linux_practice$ uname -a
Linux PALLAVI 6.18.40.1-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Fri Jul 31 22:12:15 UTC 2026 x86_64 GNU/Linux
tejup@PALLAVI:~/linux_practice$ ls
file1.txt  file2.txt  renamed.txt
tejup@PALLAVI:~/linux_practice$ ls -a
.  ..  file1.txt  file2.txt  renamed.txt
tejup@PALLAVI:~/linux_practice$ id
uid=1000(tejup) gid=1000(tejup) groups=1000(tejup),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users)
tejup@PALLAVI:~/linux_practice$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G     0  1.9G   0% /usr/lib/modules/6.18.40.1-microsoft-standard-WSL2
none            1.9G  4.0K  1.9G   1% /mnt/wsl
drivers         154G  150G  4.2G  98% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.6G  955G   1% /
none            1.9G   32K  1.9G   1% /mnt/wslg
none            1.9G     0  1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  3.3M  1.9G   1% /init
none            1.9G  480K  1.9G   1% /run
none            1.9G     0  1.9G   0% /run/lock
none            1.9G     0  1.9G   0% /run/shm
none            1.9G   80K  1.9G   1% /mnt/wslg/versions.txt
none            1.9G   80K  1.9G   1% /mnt/wslg/doc
C:\             154G  150G  4.2G  98% /mnt/c
D:\             322G   11G  312G   4% /mnt/d
tmpfs           1.9G     0  1.9G   0% /tmp
```

```
D:\             322G   11G  312G   4% /mnt/d
tmpfs           1.9G     0  1.9G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs           382M   12K  382M   1% /run/user/1000
tejup@PALLAVI:~/linux_practice$ mkdir assignment
tejup@PALLAVI:~/linux_practice$ cd assignment
tejup@PALLAVI:~/linux_practice/assignment$ touch notes.txt
tejup@PALLAVI:~/linux_practice/assignment$ echo "Hello Linux" > notes.txt
tejup@PALLAVI:~/linux_practice/assignment$ cat notes.txt
Hello Linux
tejup@PALLAVI:~/linux_practice/assignment$ cp notes.txt backup.txt
tejup@PALLAVI:~/linux_practice/assignment$ mv copy.txt backup.txt
mv: cannot stat 'copy.txt': No such file or directory
tejup@PALLAVI:~/linux_practice/assignment$ cp notes.txt copy.txt
tejup@PALLAVI:~/linux_practice/assignment$ mv copy.txt backup.txt
tejup@PALLAVI:~/linux_practice/assignment$ ls -l
total 8
-rw-r--r-- 1 tejup tejup 12 Oct 10 13:23 backup.txt
-rw-r--r-- 1 tejup tejup 12 Oct 10 13:22 notes.txt
tejup@PALLAVI:~/linux_practice/assignment$ wc -w notes.txt
2 notes.txt
tejup@PALLAVI:~/linux_practice/assignment$ wc -l notes.txt
1 notes.txt
tejup@PALLAVI:~/linux_practice/assignment$ history
    1  sudo apt update
    2  sudo apt upgrade
    3  lsb_release -a
    4  pwd
    5  ls
    6  ls -l
    7  ls -a
    8  mkdir linux practice
    9  cd linux-practice
   10  pwd
   11  clear
   12  ls
```

```
14  ls
15  cd linux-practice
16  pwd
17  cd __
18  cd ..
19  pwd
20  mkdir projects
21  mkdir documents
22  mkdir practice
23  ls
24  touch hello.txt
25  ls
26  touch file1.text file2.text file3.text
27  ls
28  echo "Hello linux" > hello.text
29  cat hello.txt
30  cat hello.text
31  echo "Second" >> hello.text
32  cat hello.text
33  touch original.txt
34  cp original.txt copy.txt
35  ls
36  pwd
37  ls
38  ls -l
39  ls -a
40  whoami
41  date
42  cal
43  mkdir linux_practice
44  touch file1.txt
45  echo "Hello Linux"
46  echo "Linux Practice" > file1.txt
47  cat file1.txt
48  cp file1.txt file2.txt
49  mv file1.txt renamed.txt
50  wc -l file1.txt
51  touch file1.txt
```

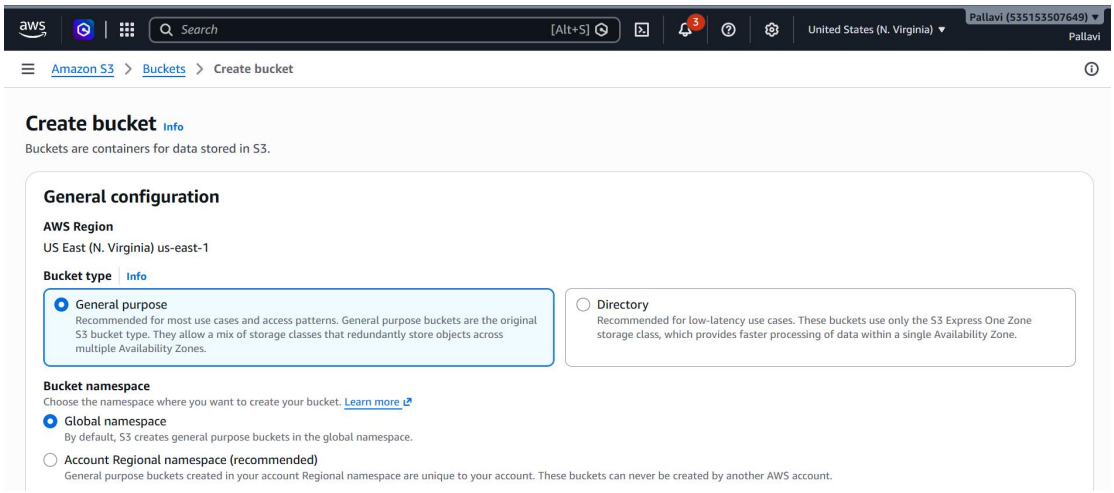
```
   55  cat file1.txt
   56  wc -l file1.txt
   57  head -n 5 file1.txt
   58  free -h
   59  uname -a
   60  uname -a\
   61  uname -a
   62  df -h
   63  wc -w notes.txt
   64  clear
   65  pwd
   66  whoami
   67  hostname
   68  date
   69  uname -a
   70  ls
   71  ls -a
   72  id
   73  df -h
   74  mkdir assignment
   75  cd assignment
   76  touch notes.txt
   77  echo "Hello Linux" > notes.txt
   78  cat notes.txt
   79  cp notes.txt backup.txt
   80  mv copy.txt backup.txt
   81  cp notes.txt copy.txt
   82  mv copy.txt backup.txt
   83  ls -l
   84  wc -w notes.txt
   85  wc -l notes.txt
   86  history
tejup@PALLAVI:~/linux_practice/assignment$
```

AWS Account / Region:

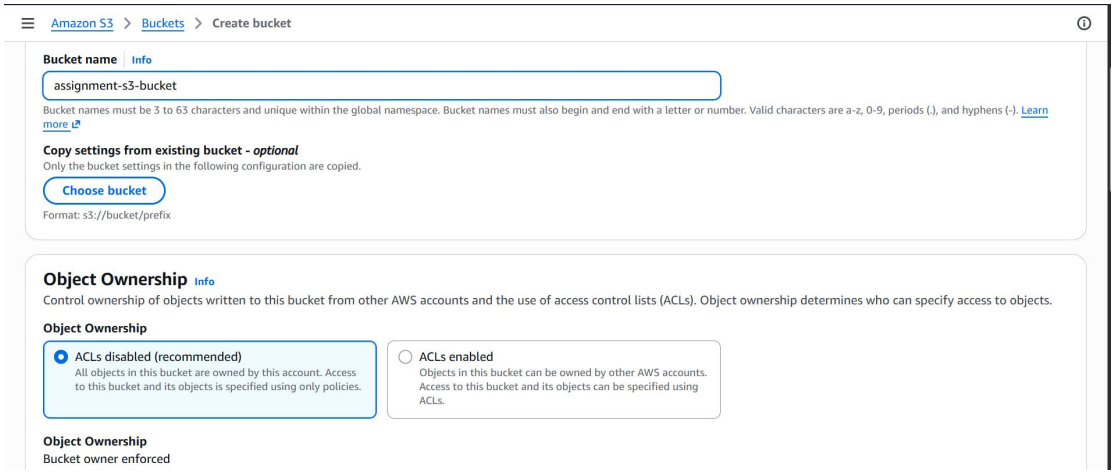


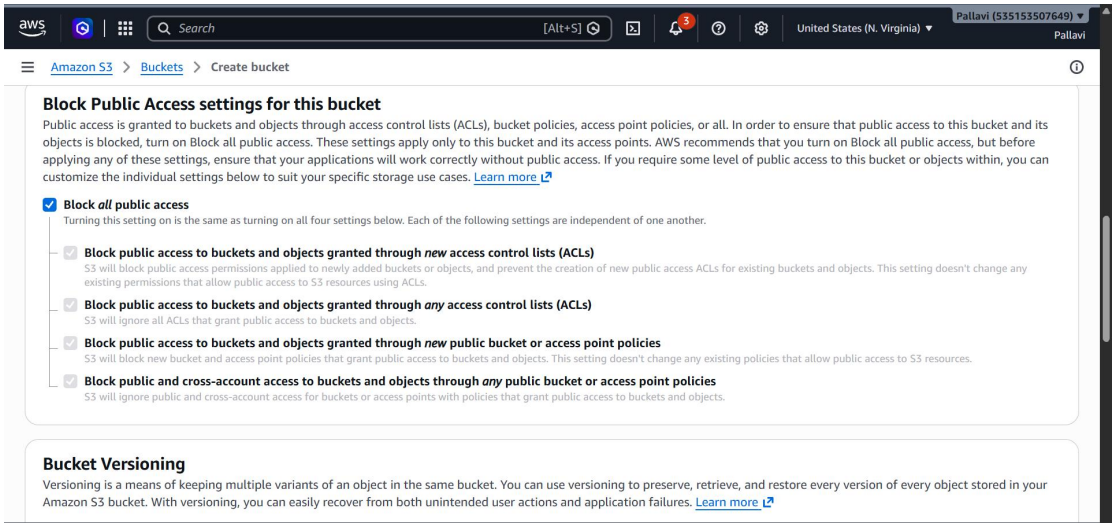
Amazon S3:

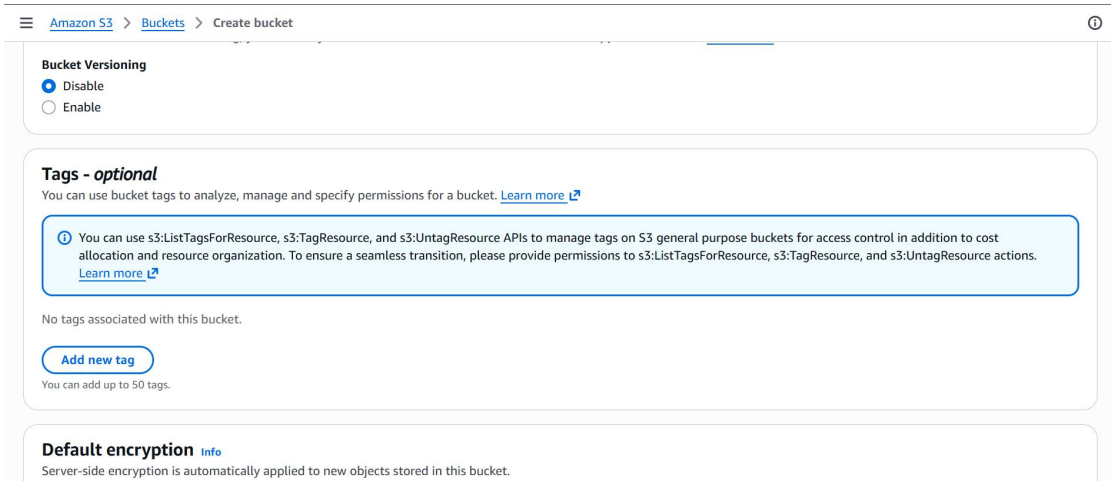
Create an S3 bucket and keep the bucket type as general purpose

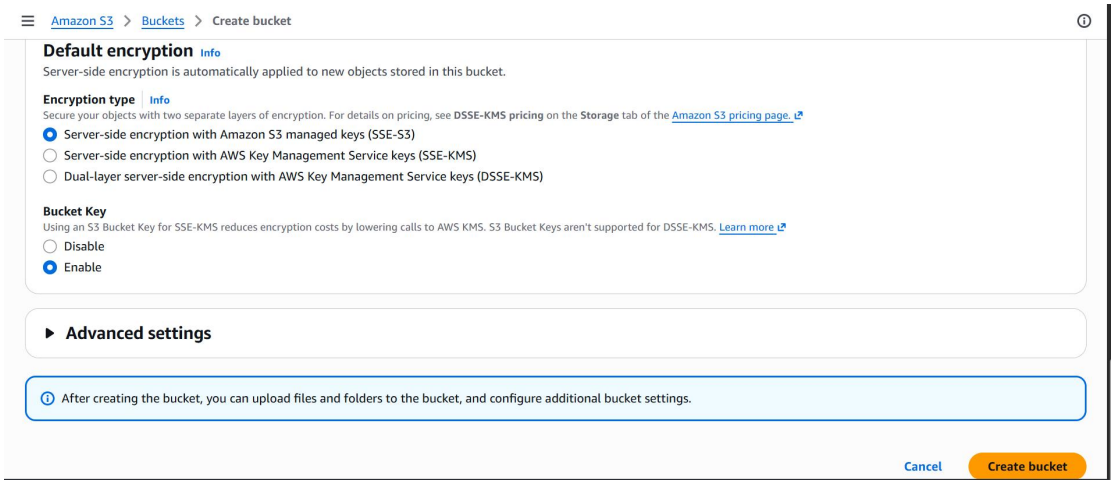


Enter a unique bucket name

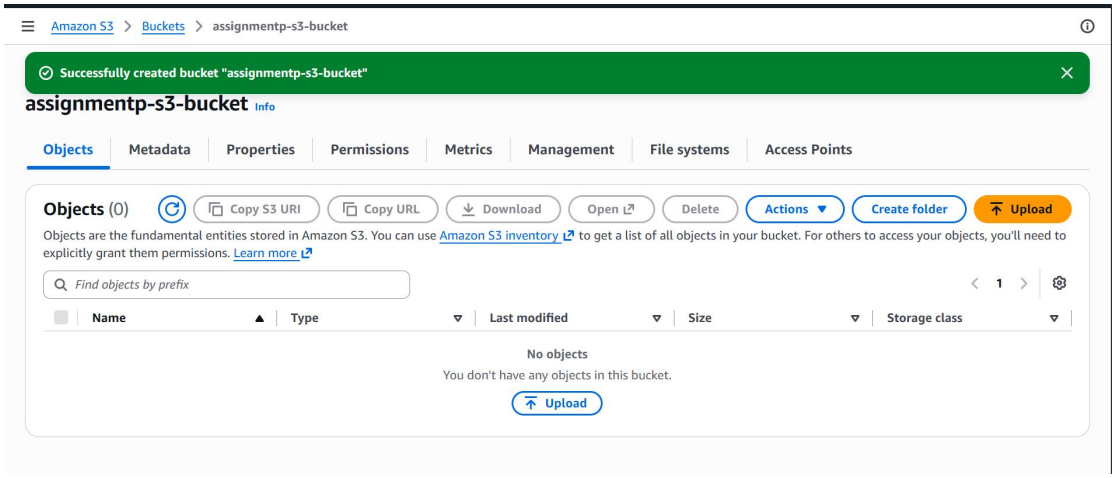




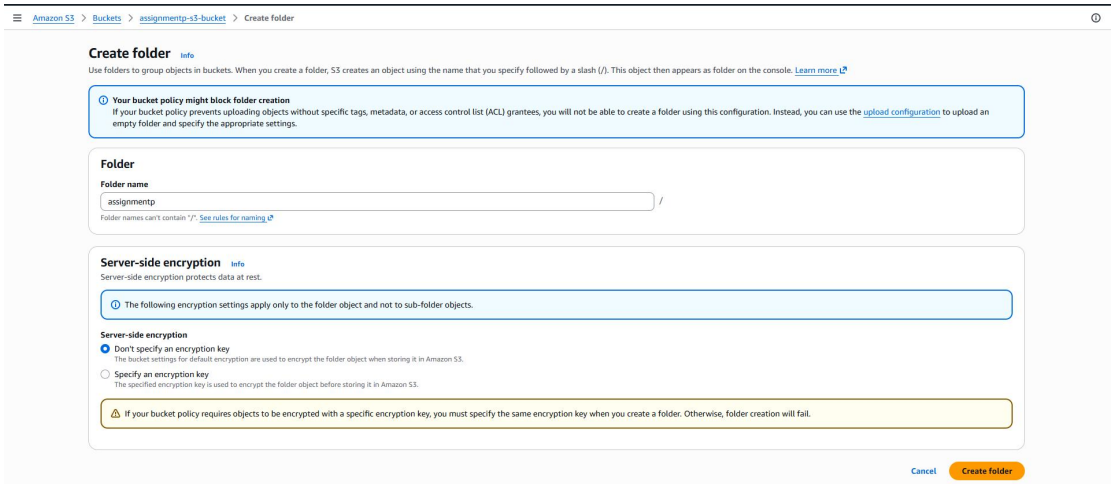


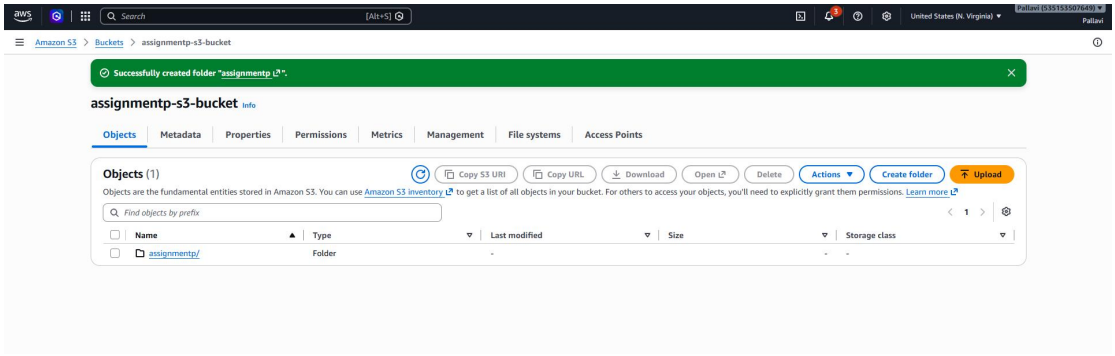


Bucket is created successfully and now appears empty with 0 objects

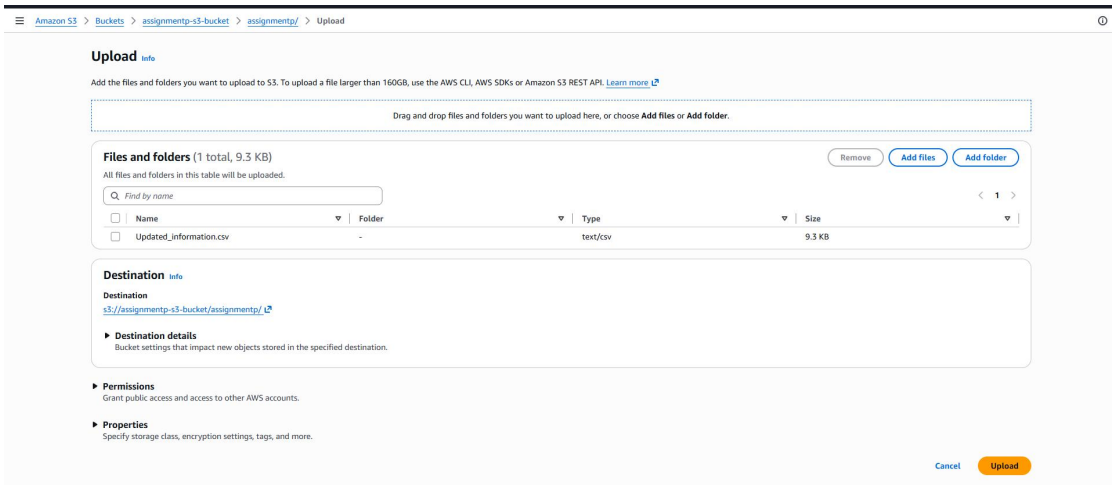


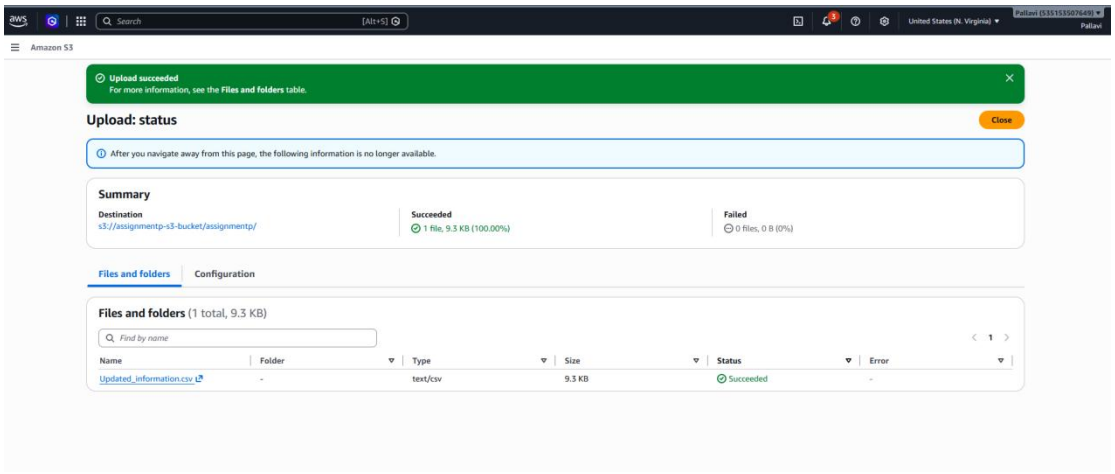
Create a folder

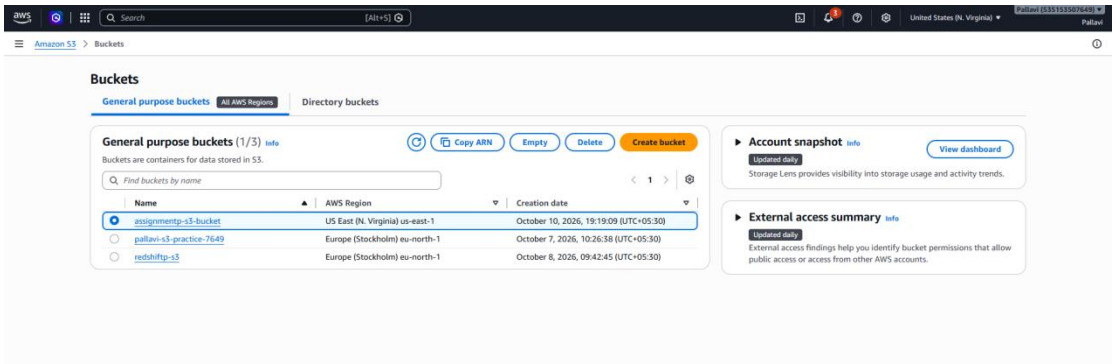




Upload the file

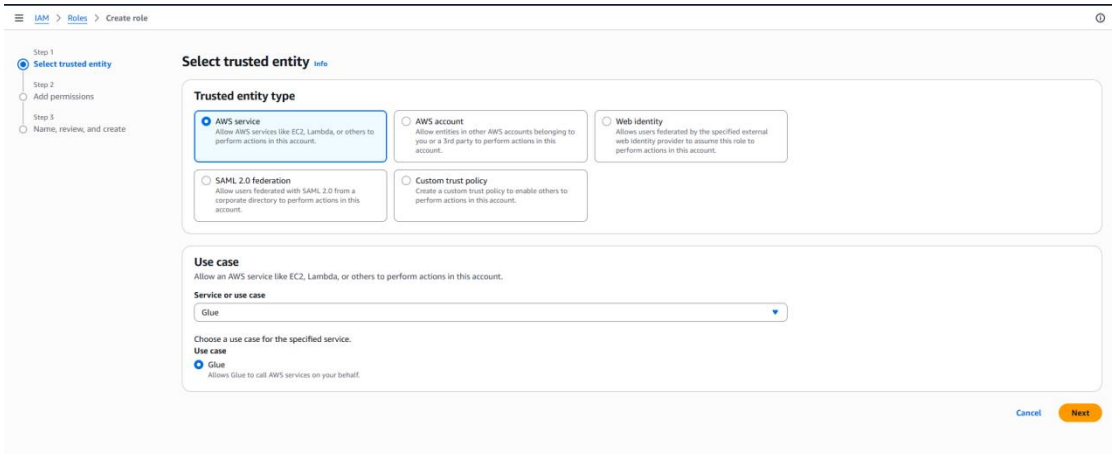




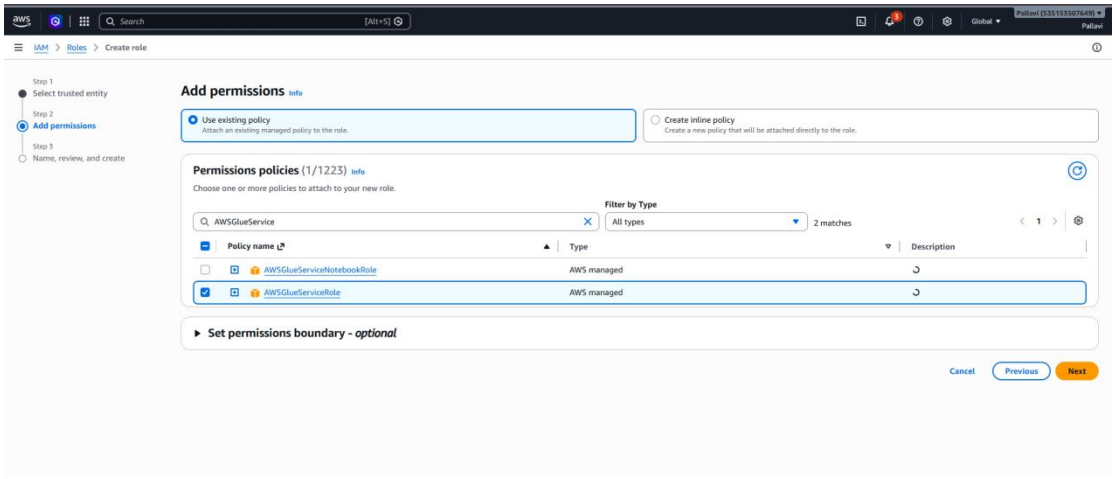


IAM:

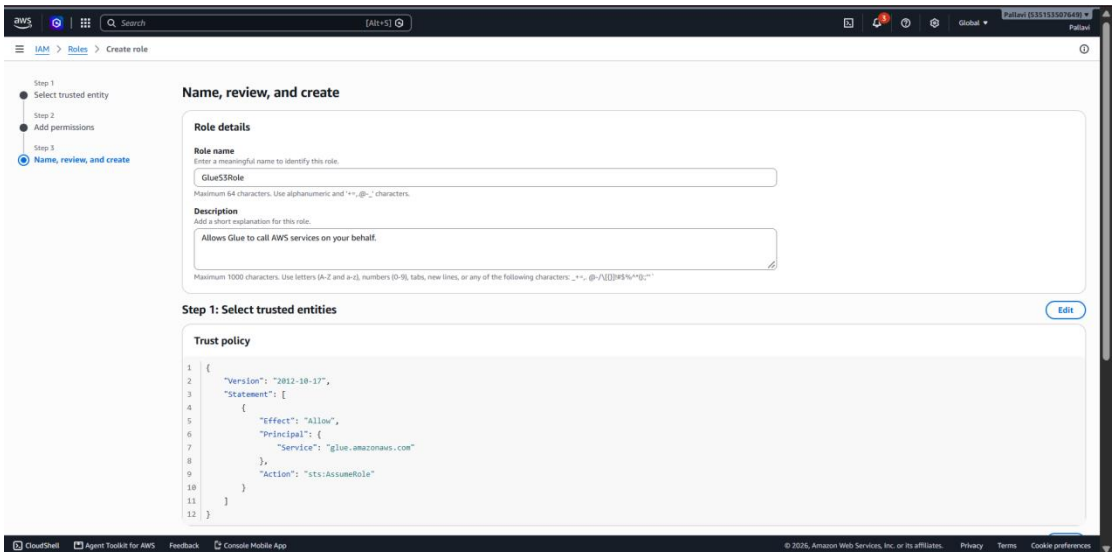
Select trusted entity as AWS service

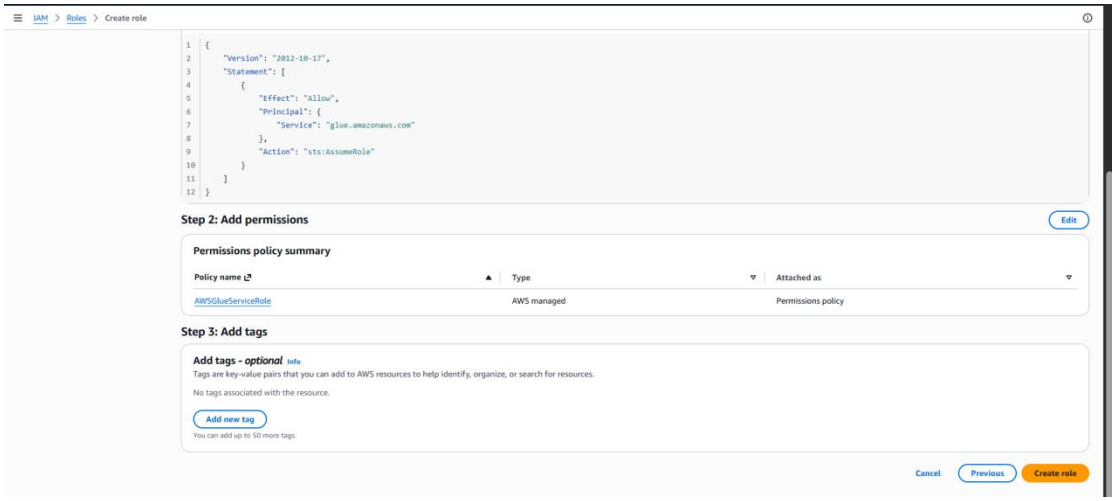


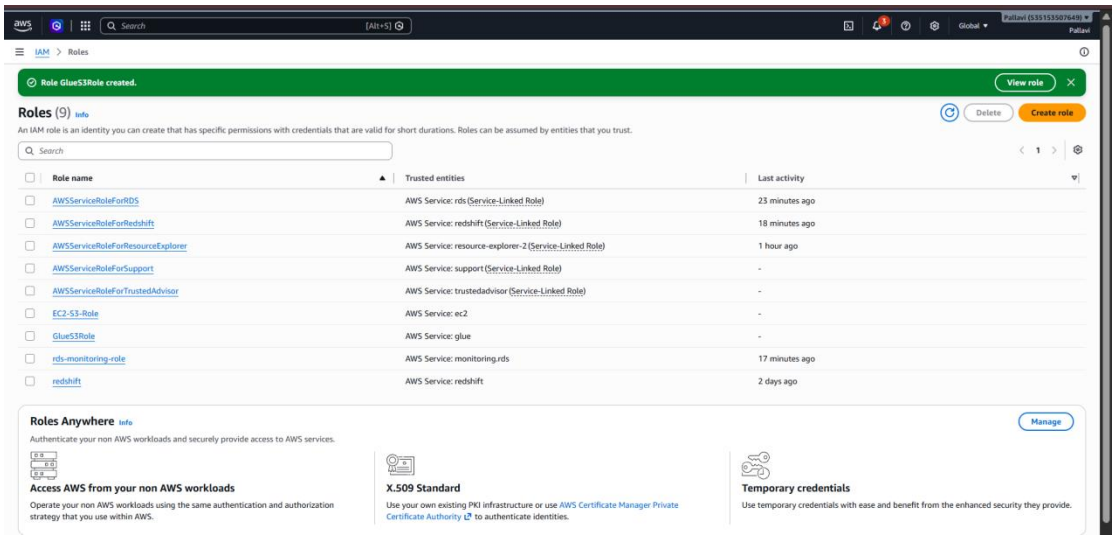
Add permissions



Role name

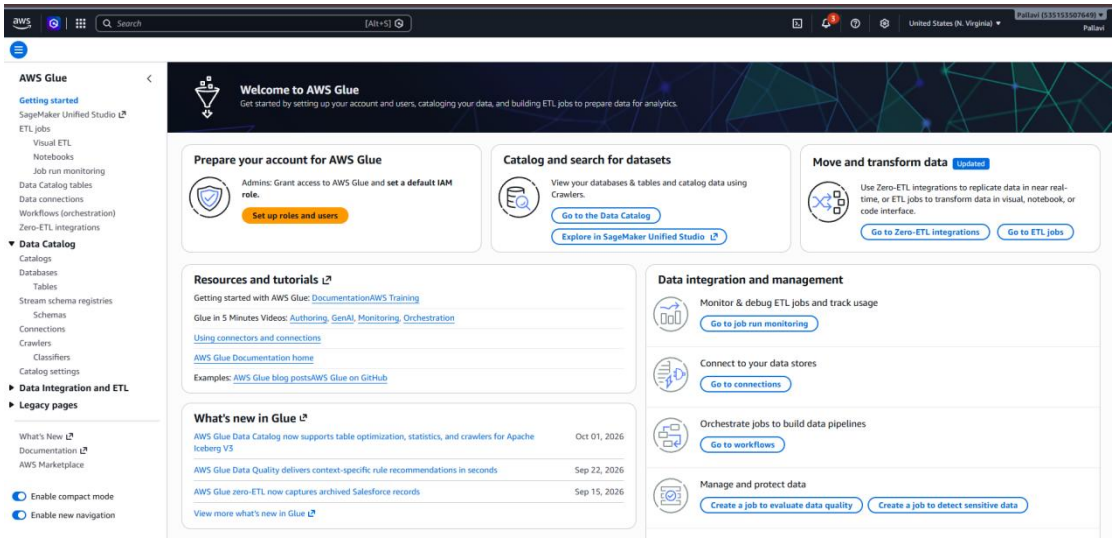




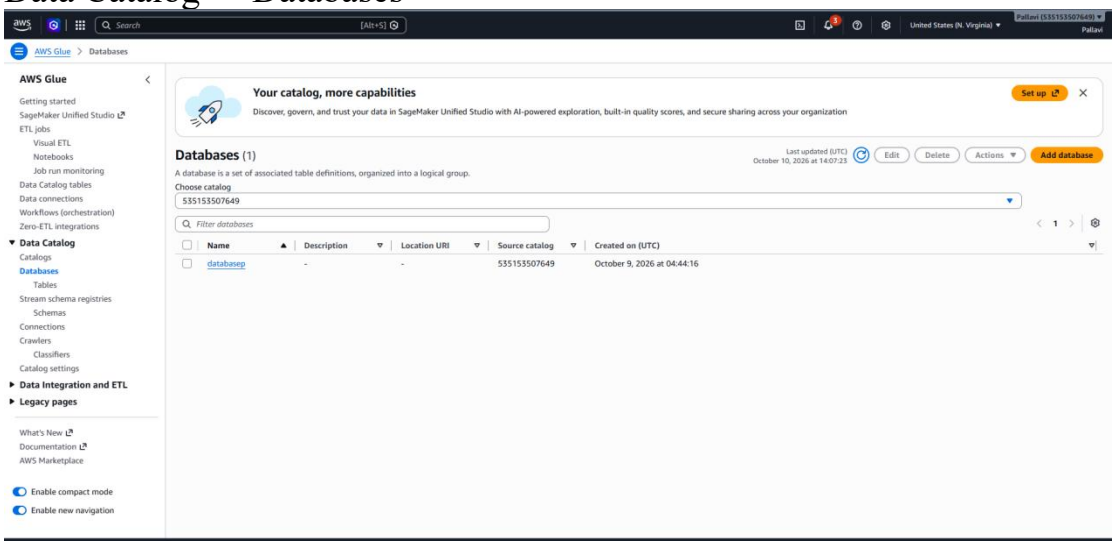


GLUE:

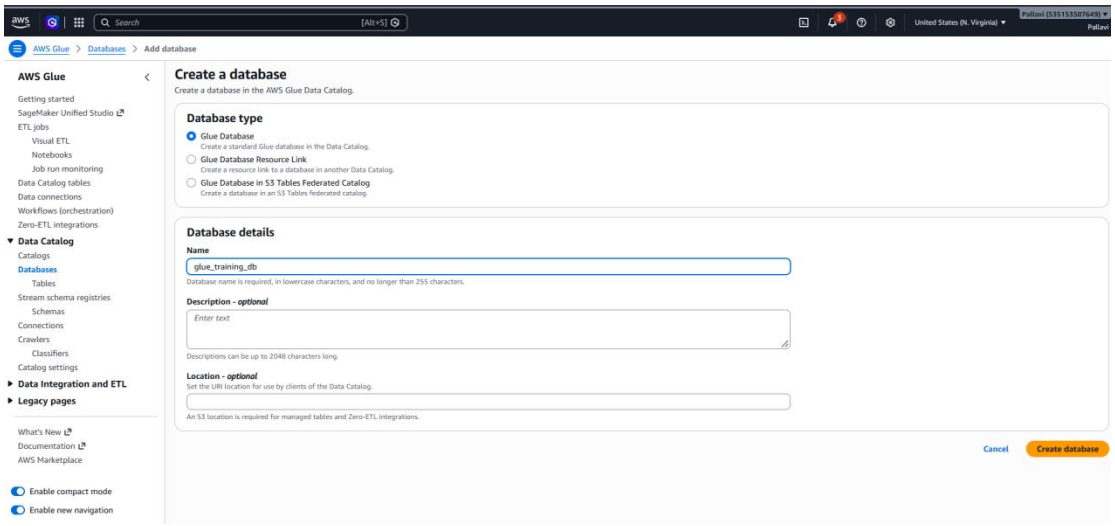
AWS Glue Console



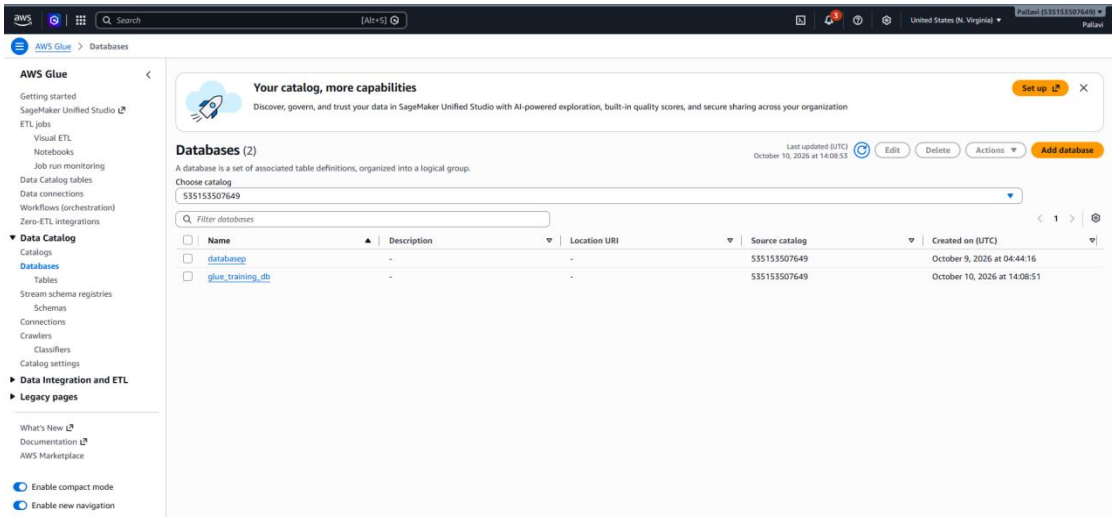
Data Catalog -> Databases



Create a database

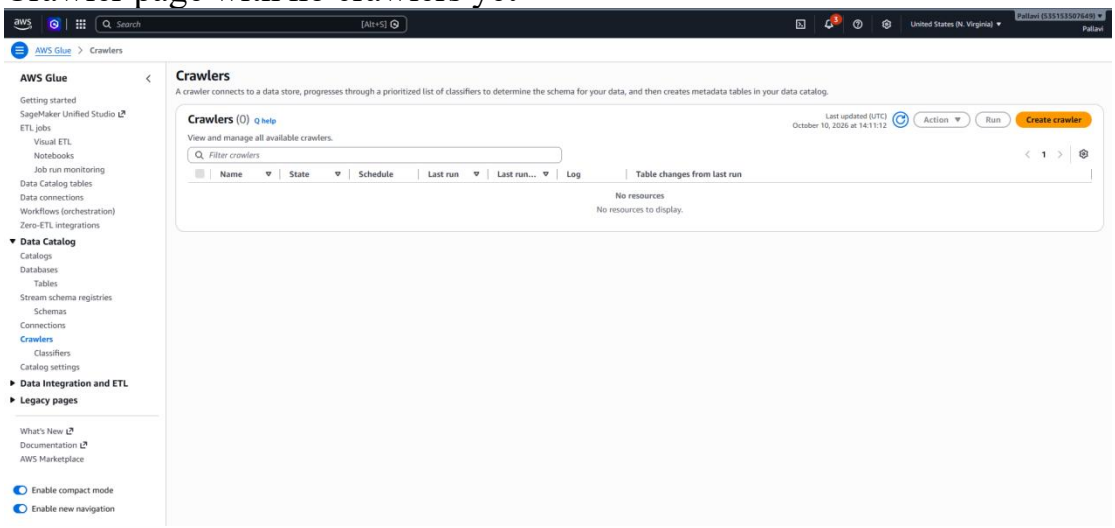


Database list now showing glue_training_db

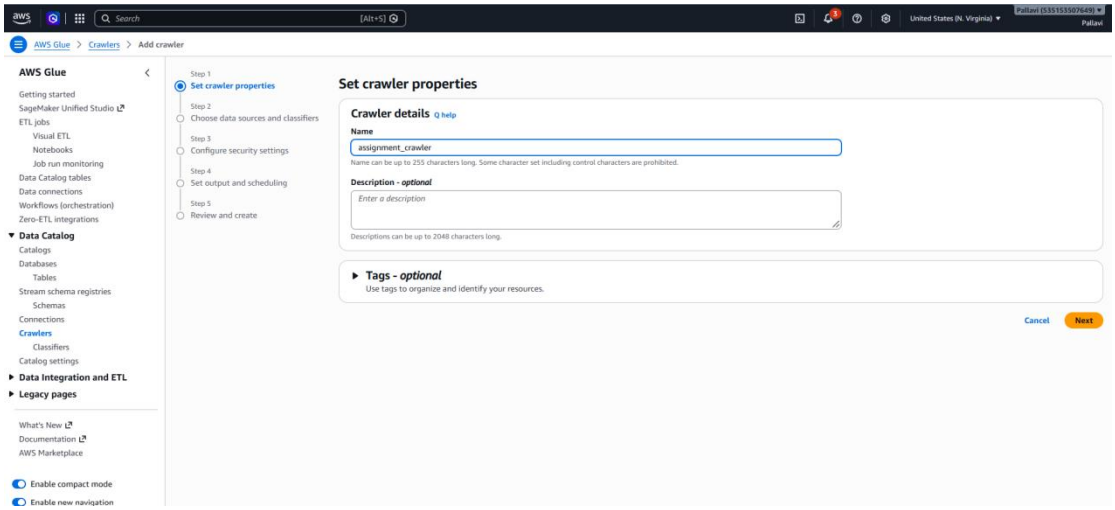


AWS CRAWLER

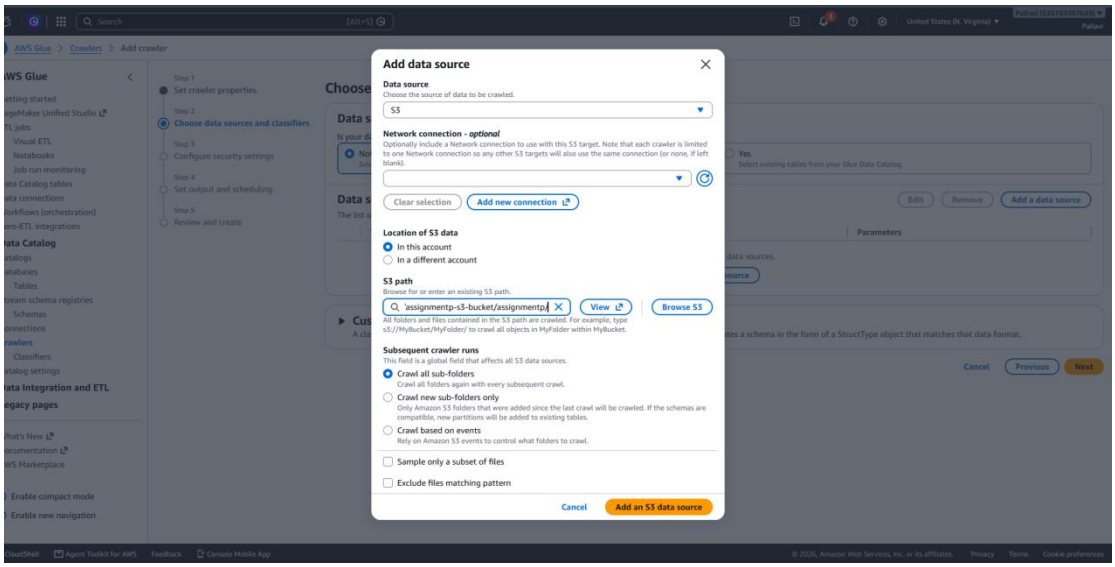
Crawler page with no crawlers yet



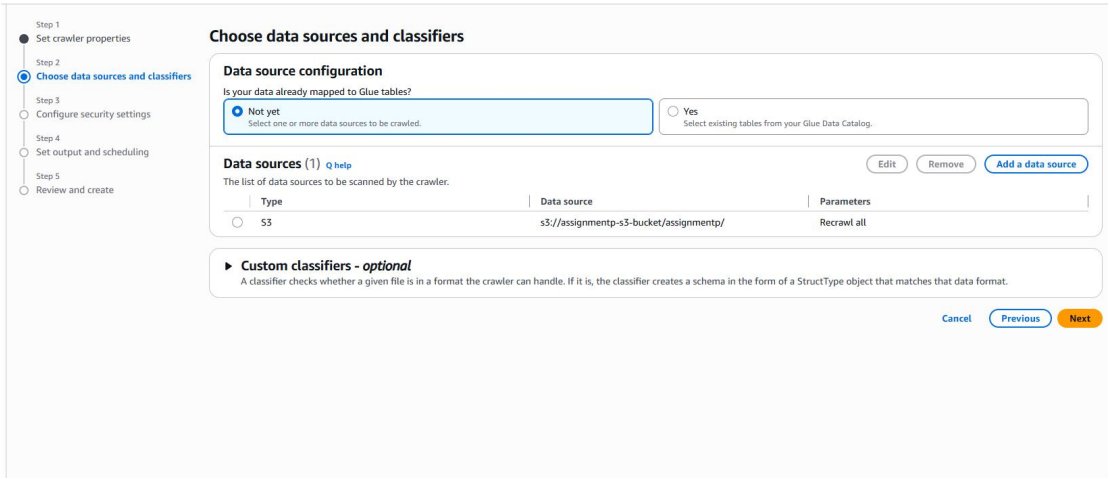
Crawler properties page with the crawler name entered as assignment_crawler.



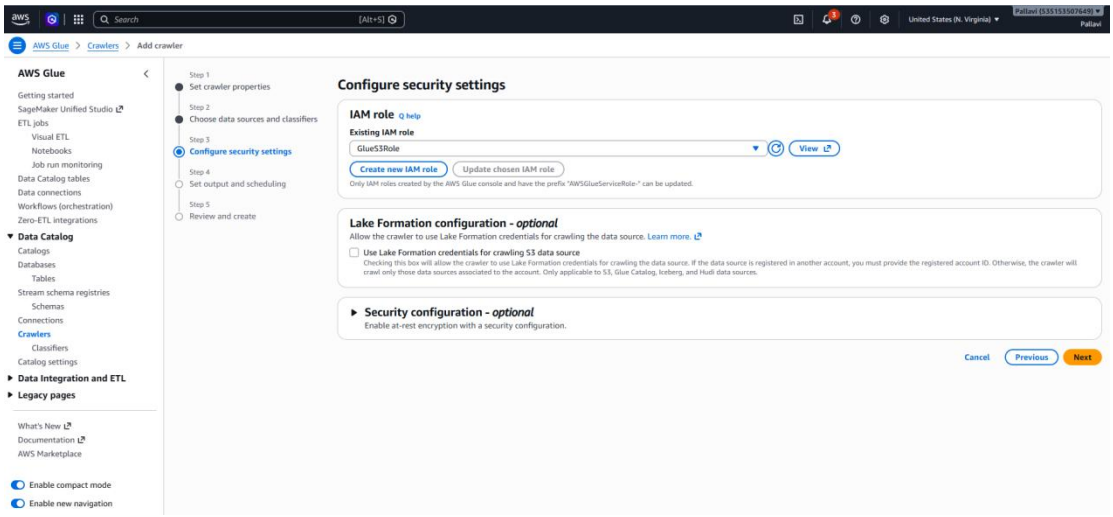
S3 source location



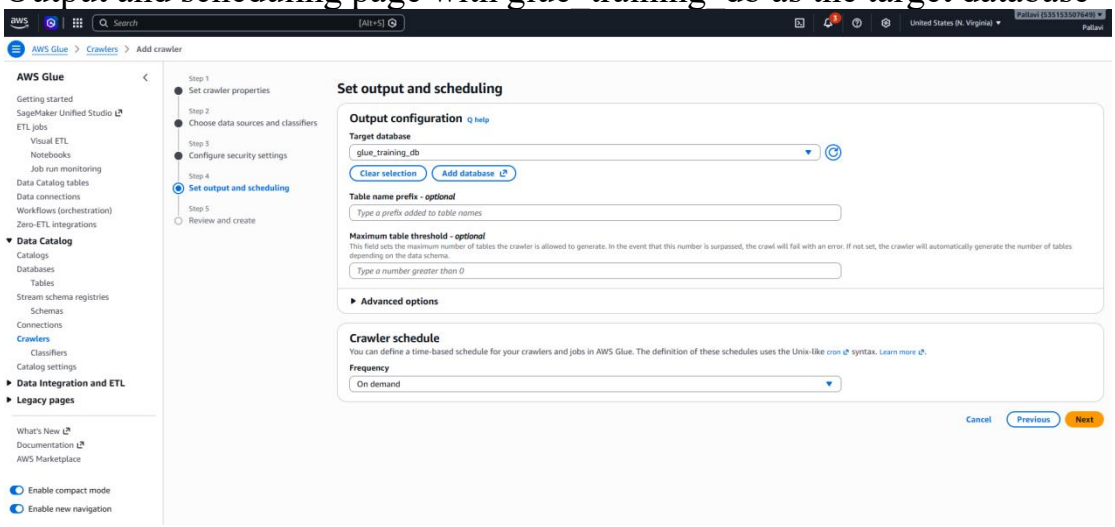
Data Source added



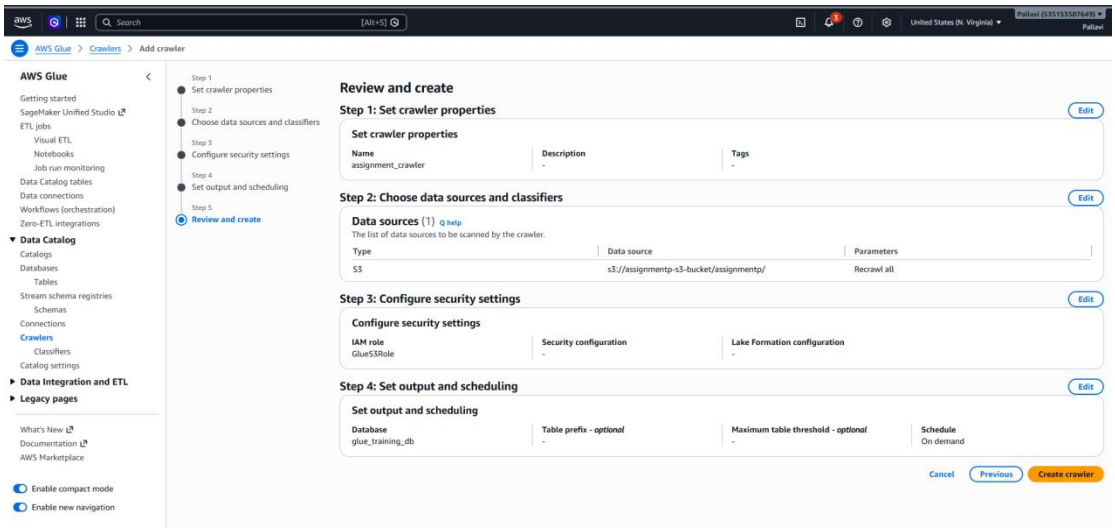
Configure security settings page with IAM role

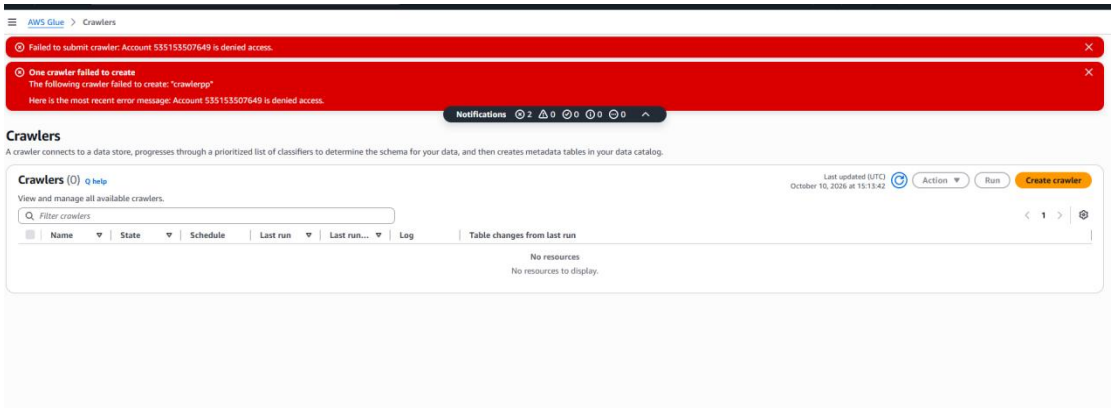


Output and scheduling page with glue_training_db as the target database



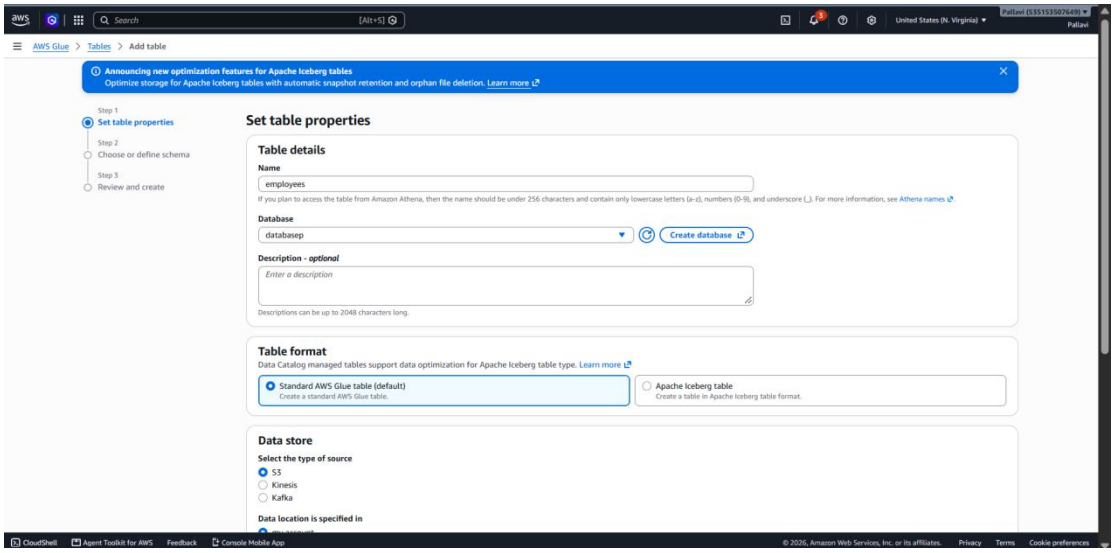
Review and create

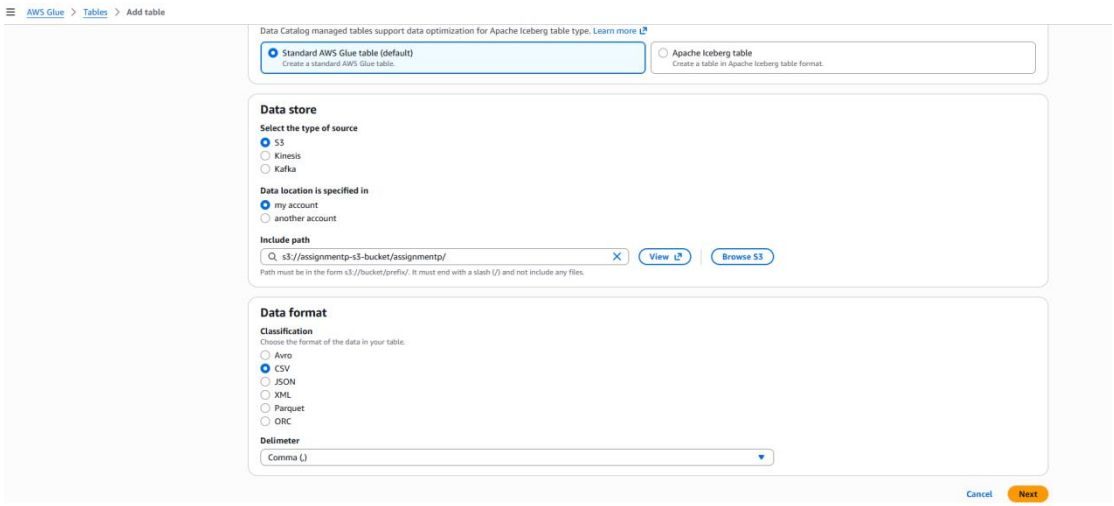


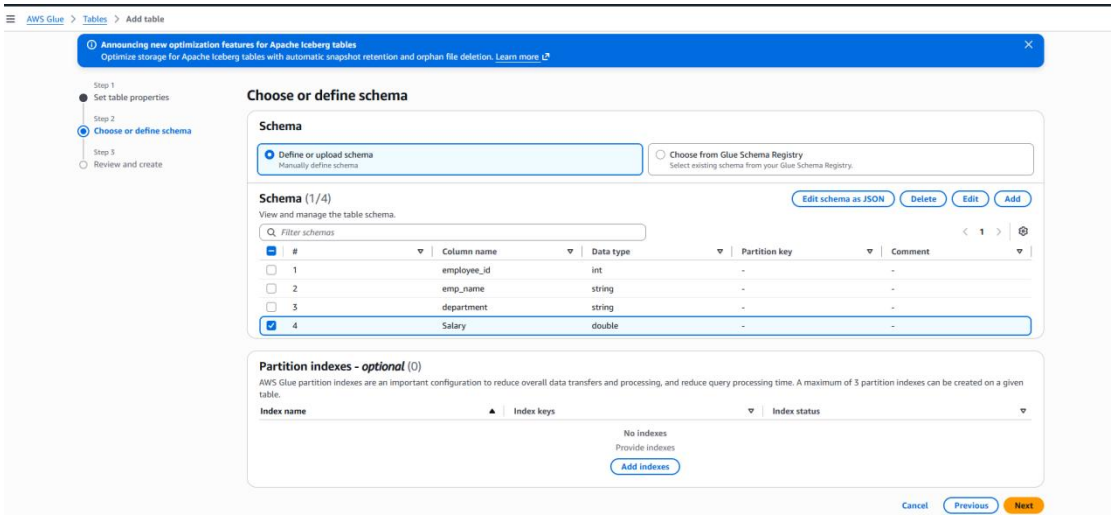


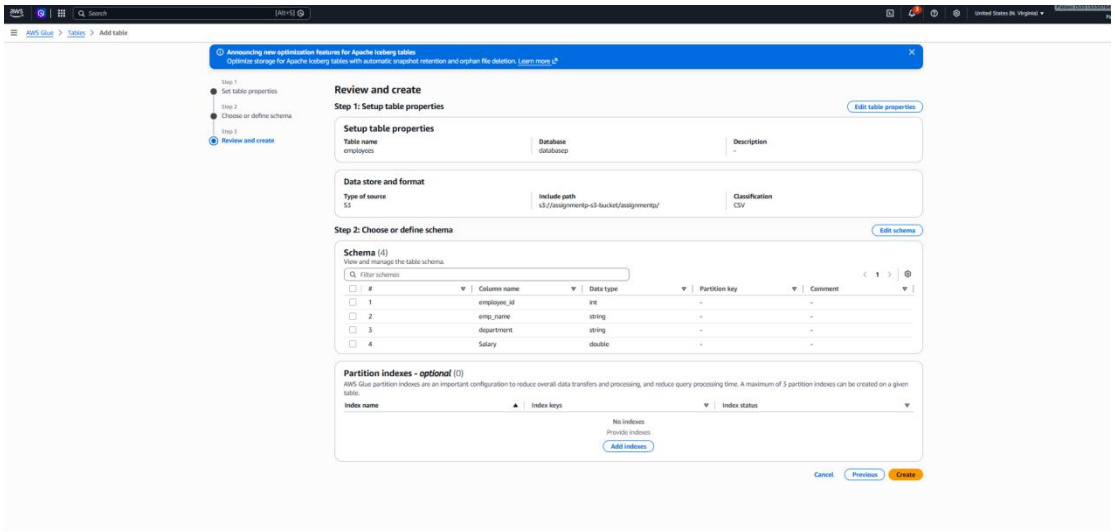
GLUE DATA CATALOG

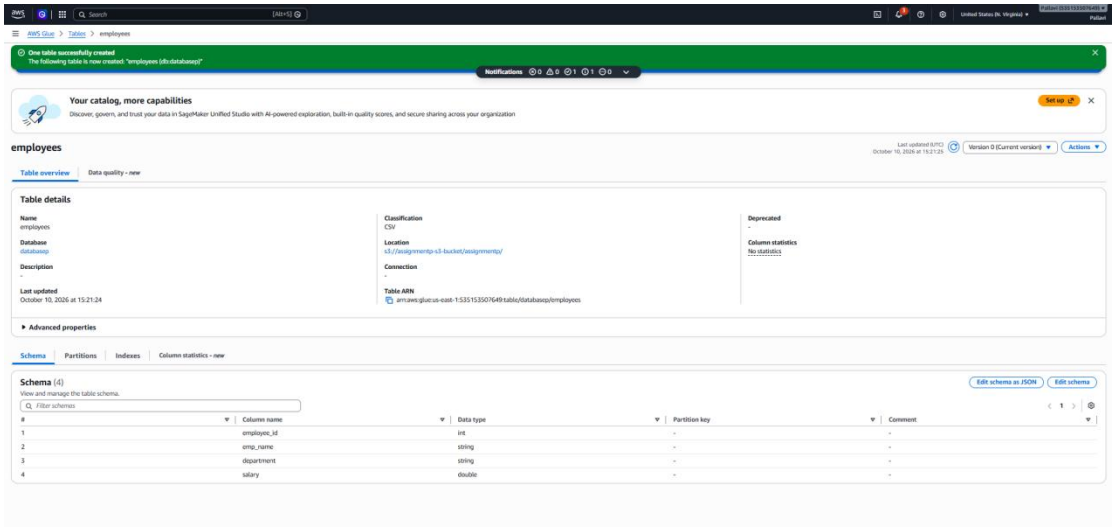
Set table properties



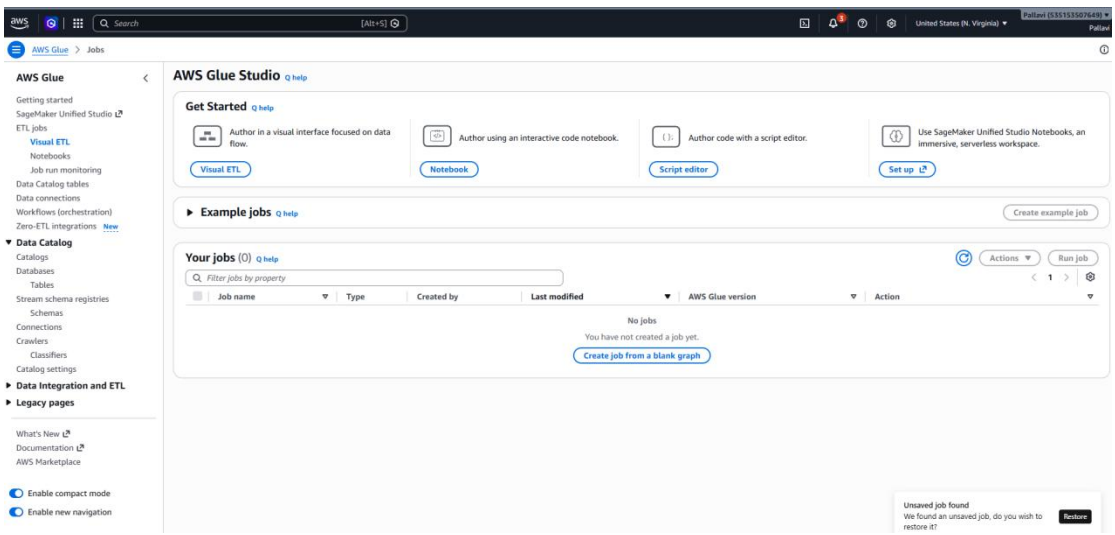


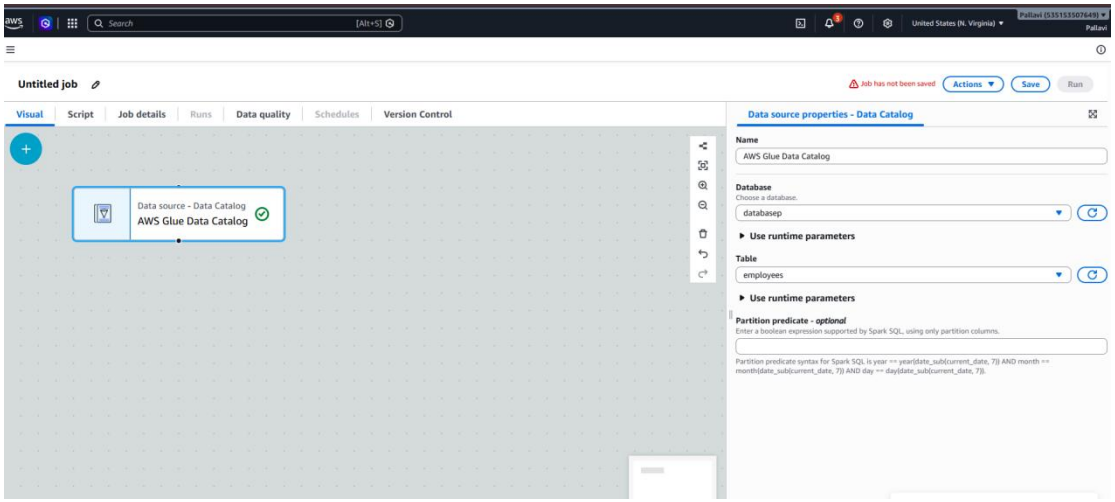


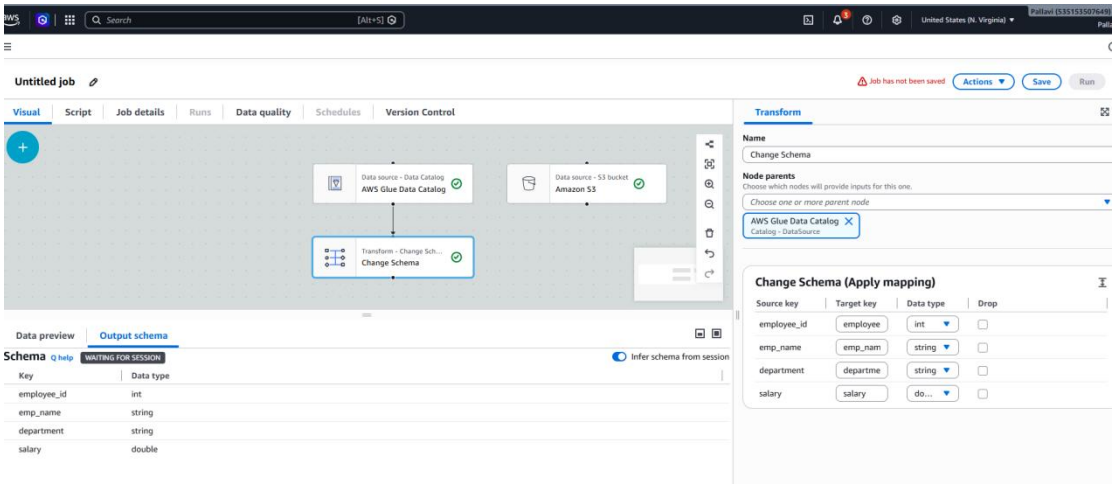


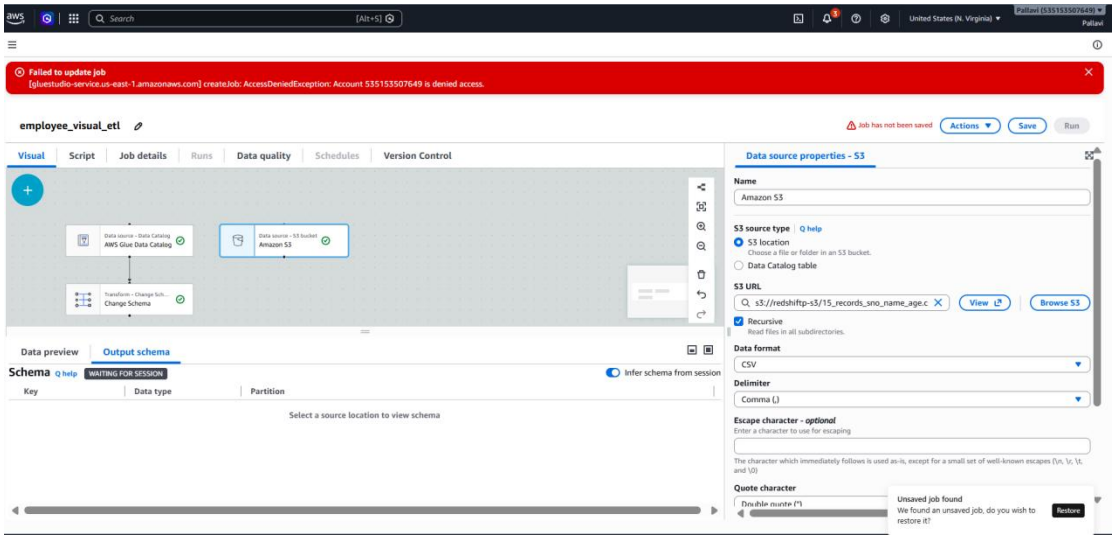


GLUE VISUAL ETL

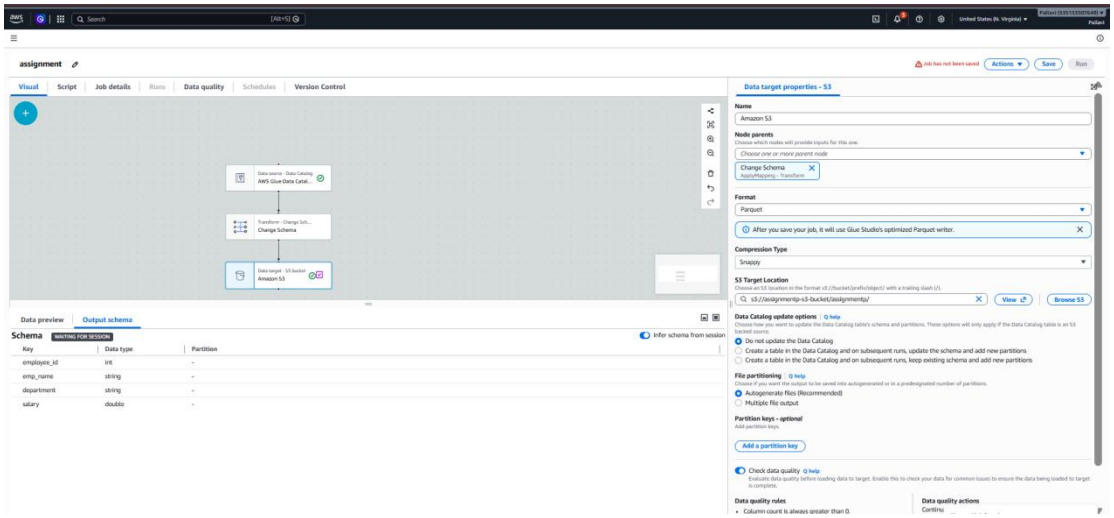


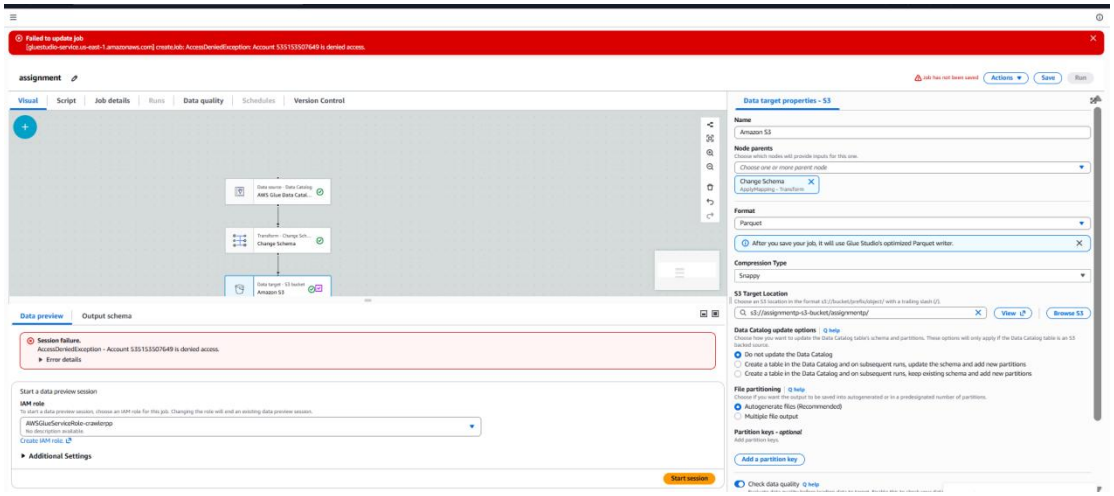

Untitled job
Job has not been saved
Actions
Save
Run
Visual
Script
Job details
Runs
Data quality
Schedules
Version Control
Data source - Data Catalog
AWS Glue Data Catalog
Data source properties - Data Catalog
Name
AWS Glue Data Catalog
Database
Choose a database.
databasep
Use runtime parameters
Table
employees
Use runtime parameters
Partition predicate - optional
Enter a boolean expression supported by Spark SQL, using only partition columns.


Untitled job
Job has not been saved
Actions
Save
Run
Visual
Script
Job details
Runs
Data quality
Schedules
Version Control
Data source - Data Catalog
AWS Glue Data Catalog
Data source - S3 bucket
Amazon S3
Transform - Change Sch...
Change Schema
Transform
Name
Change Schema
Node parents
Choose which nodes will provide inputs for this one.
Choose one or more parent node
AWS Glue Data Catalog
Catalog - DataSource
Change Schema (Apply mapping)
Source key
Target key
Data type
Drop
employee_id
employee
int
emp_name
emp_nam
string
department
departme
string
salary
salary
do...
Data preview
Output schema
Schema
WAITING FOR SESSION
Infer schema from session
Key
Data type
employee_id
int
emp_name
string
department
string
salary
double


Failed to update job
[gluestudio-service.us-east-1.amazonaws.com] createJob: AccessDeniedException: Account 535153507649 is denied access.
employee_visual_etl
Job has not been saved
Actions
Save
Run
Visual
Script
Job details
Runs
Data quality
Schedules
Version Control
Data source properties - S3
Name
Amazon S3
S3 source type
S3 location
Choose a file or folder in an S3 bucket.
Data Catalog table
S3 URL
s3://redshiftp-s3/15_records_sno_name_age.c
View
Browse S3
Recursive
Read files in all subdirectories.
Data format
CSV
Delimiter
Comma (,)
Escape character - optional
Enter a character to use for escaping
Quote character
Unsaved job found
We found an unsaved job, do you wish to restore it?
Restore
Data preview
Output schema
Schema
WAITING FOR SESSION
Infer schema from session
Key
Data type
Partition
Select a source location to view schema

S3 Target Location





Final Explanation : AWS Glue ETL Workflow

1. Amazon S3:

Amazon S3 is used to store data.
First, we upload the employee data file to an S3 bucket.
The file contains details of employees.
We store the raw data in the input folder.
This data is used in the next steps of the workflow.

2. Glue Crawler

A Glue Crawler is used to scan the data stored in S3.
It checks the file and identifies its structure.
It finds the column names and data types.
It collects the details about the data.I
t sends these details to the Glue Data Catalog.

3. Glue Data Catalog

The Glue Data Catalog stores information about the data.
It contains databases and tables.
We create a database and an employees table.
The table stores details about the columns in the employee data.
Glue uses this table to read the source data.

4. Glue Visual ETL

Glue Visual ETL is used to create a data workflow.
We create the workflow using boxes on the screen.
We select the employees table as the data source.
We connect the source to a transformation and an S3 target.
This helps us move and change the data without writing the whole code manually.

5. Transformation

Transformation is used to make changes to the data.I
In this workflow, we use the Change Schema option.
We select the required employee columns.
We can change the column names or data types if needed.
The changed data is then sent to the S3 target.

6. S3 Curated Data

The transformed data is stored in the S3 curated folder.
We set the output location as s3://glue-ass-p/curated/.
We select Parquet as the output format.
When the Glue job runs successfully, it creates output files in this folder.
We can use these files for further analysis and reporting.

WORKFLOW : s3 -> Glue Crawler -> Glue Data Catalog -> Glue Visual ETL -> Transformation -> S3 Curated data